

Milestone-III

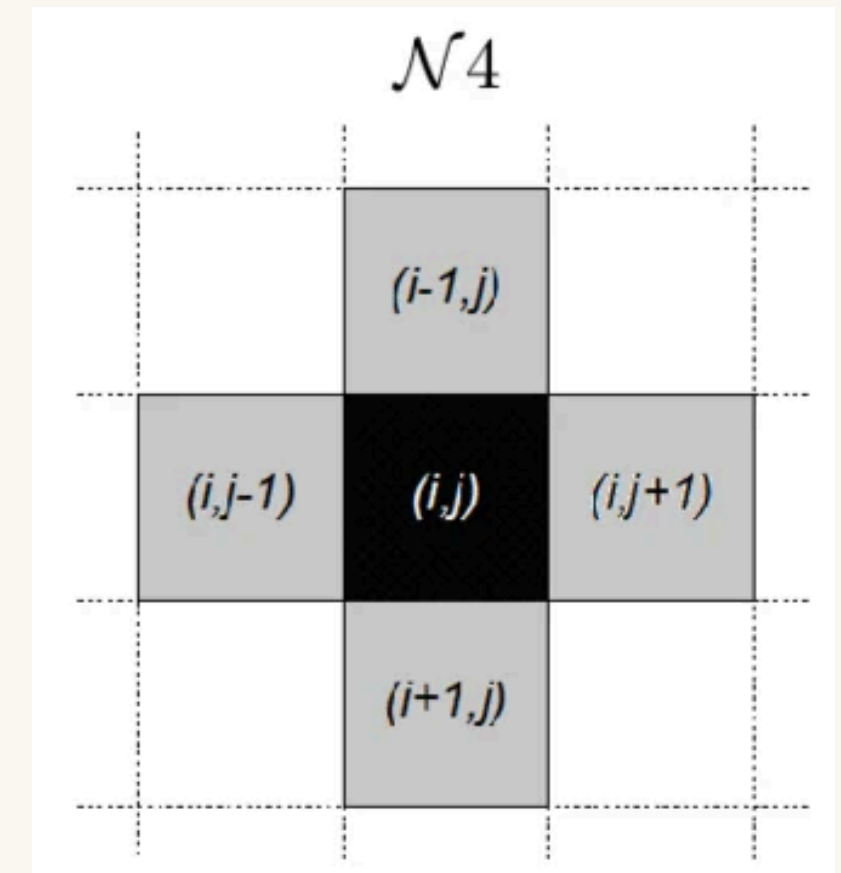
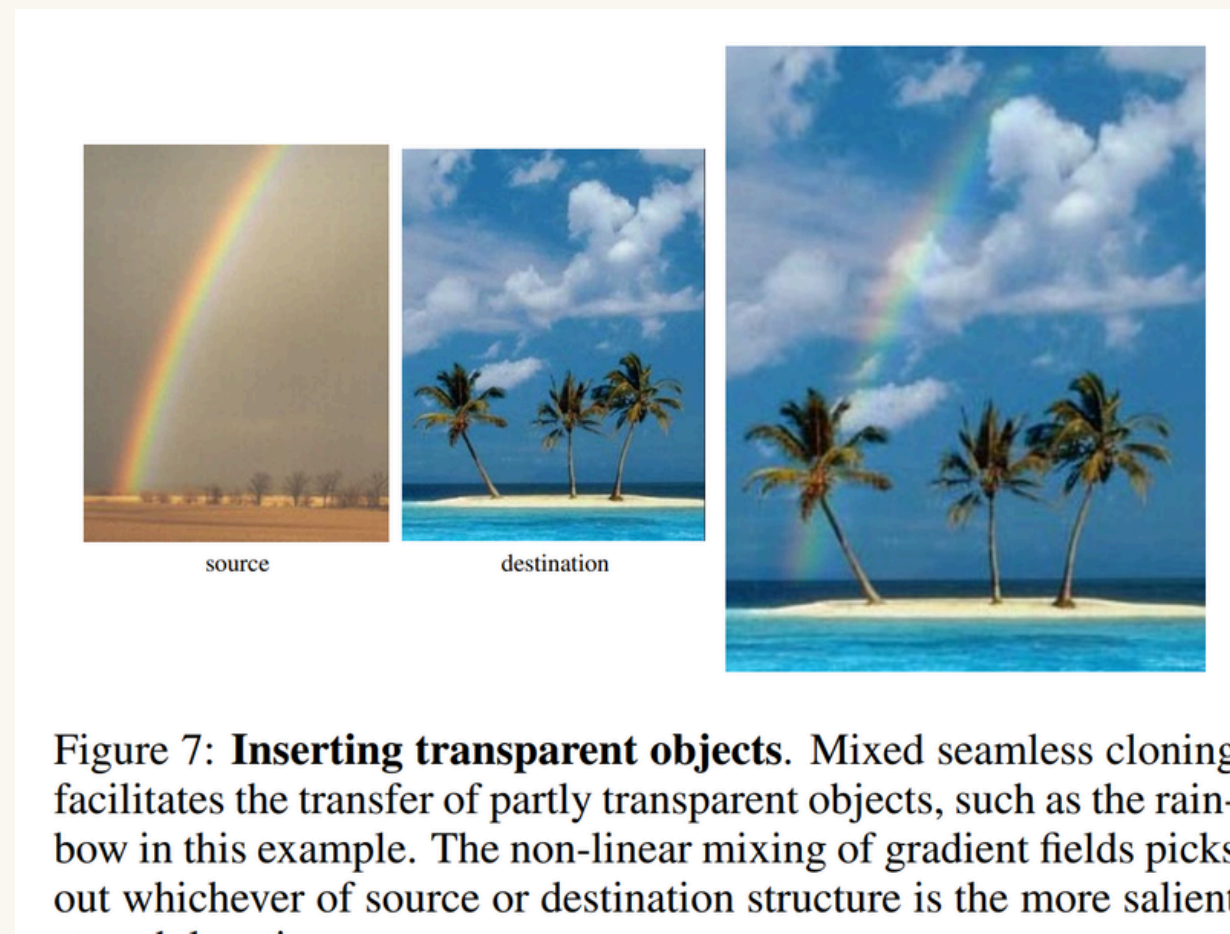
Fast Poisson Image Editing

TEAM MEMBERS

Fatima Sohail (23I-0633)
Menahil Ahmad (23I-0546)
CS-D

Recap of Fast Poisson Image Editing

- Solves the problem of visible seams when pasting a source image region onto a target image by using **gradients for blending**



Jacobi Poisson Equation

$$x_p^{(k+1)} = \frac{1}{4} \left(\text{div } \mathbf{v}_p + x_{\text{up}}^{(k)} + x_{\text{down}}^{(k)} + x_{\text{left}}^{(k)} + x_{\text{right}}^{(k)} \right)$$

This is a **memory bound** problem

Each pixel is accessed **5 TIMES!**

Project Scope

- 1. Only parallelism using OpenMP is considered.
- 2. The workload → 4,291 to 1,052,676 masked pixels (square6–10 and circle6–10 from the FPIE benchmark suite).
- 3. Single-node execution on x86-64 CPU architectures.

Implementation Details

Languages	Compiler Flags	Tools
• C++ • Python 3.12.3	• -std=c++11: • -O3 • fopenmp	• OpenMP • CMake • GCC/G++ • perf

- Machine:
- Ubuntu 24.04.3 LTS
 - Intel(R) Core(TM) i7-8550U CPU (1.80GHz)
 - 8 CPUs, 4 cores
 - Available RAM: 7.9 GiB

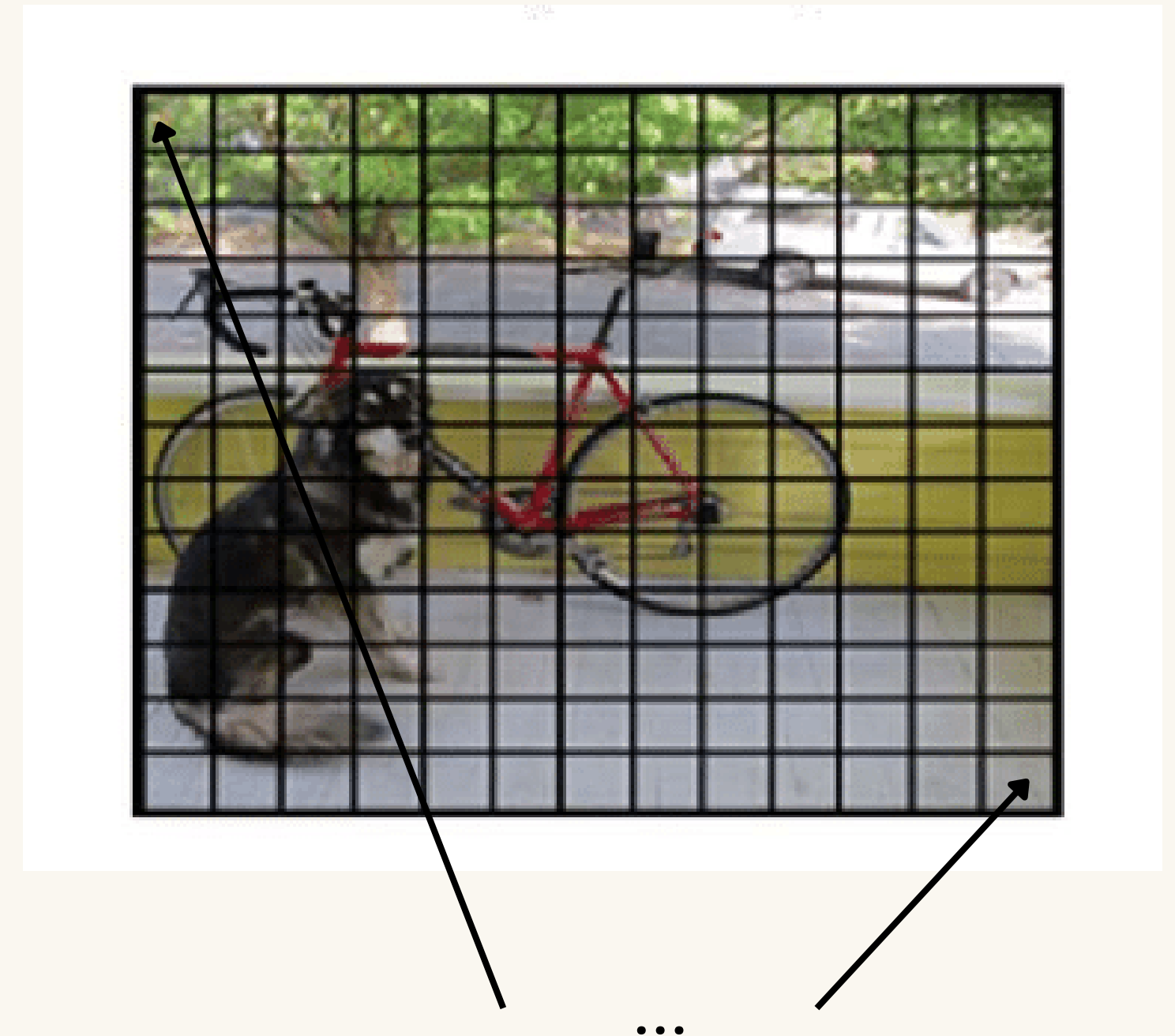
GridSolver

Basic Tiling Approach:

Divide the image into blocks (grid-x by grid-y) → tuneable

In each Jacobi Iteration:

- Compute an iteration for blocks in parallel
- Each thread updates all points in its assigned block
- Copy from temporary to target buffer ... introduces double buffering



EquSolver

Divide pixels into 2 folds:

$$(x + y) \% 2$$

Even Pixels → Red

Odd Pixels → Black

Such that each pixel only depends on pixels from the opposite coloured group

Two Phase Update:

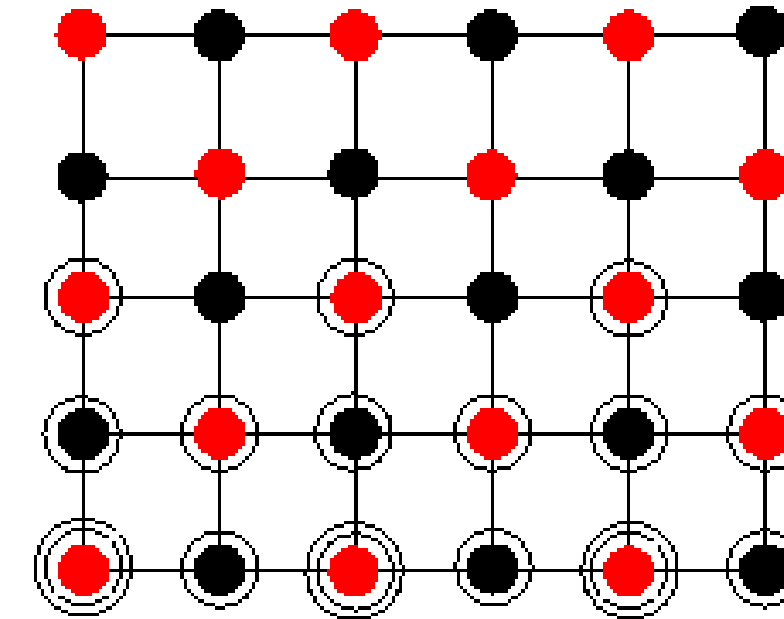
- 1.Parallel Update of Black Pixels
- 2.Parallel Update of Red Pixels

Gauss Seidel

Red Pixels update on the basis of new values of Black Pixels

Theoretical 2x faster convergence

Step 1



Flatten the image to match phases:
Black then Red pixels

1...n_mid : Black

n_mid...n: Red

Block Red Black & Multiple Sweeps Red-Black Solver

BrbSolver

- A hybrid solver of EquSolver and GridSolver
- Color tiles red or black instead of pixels
- Apply Gauss Seidel on target image buffer → no double buffering

MsrbSolver

- An Extension of BrbSolver
- Dynamic alpha updation version of RB-Block solver

Residual Proportional Scaling

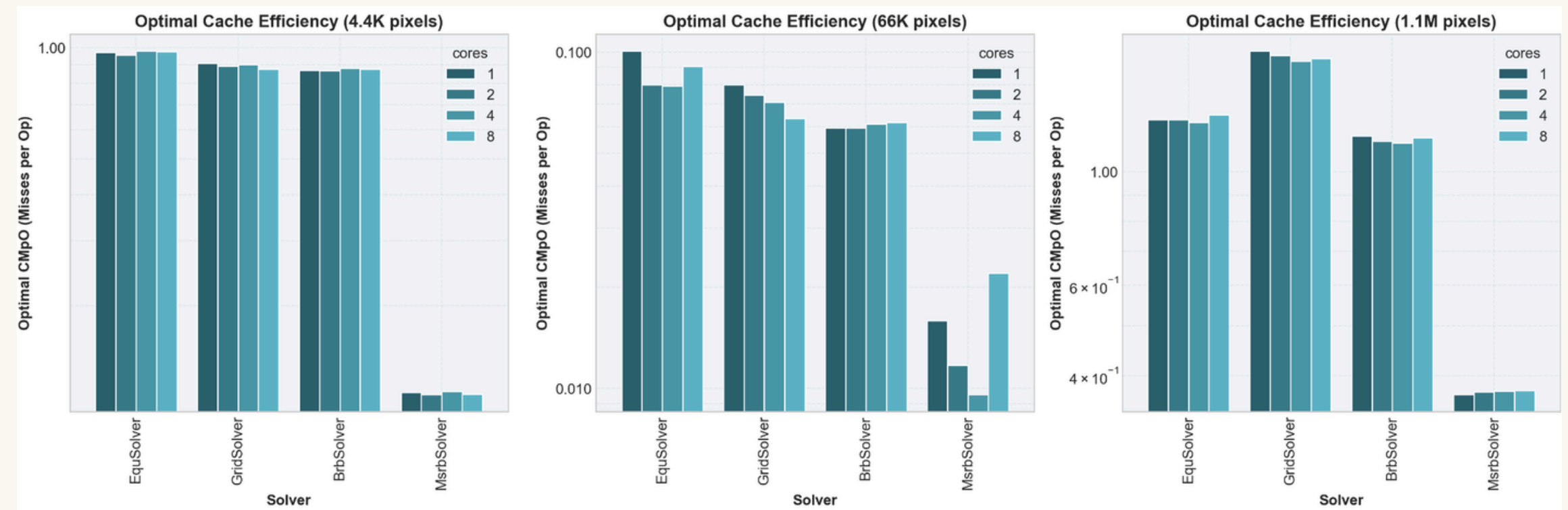
$$\alpha_k + 1 = \max(1, \text{round}(\alpha_{\max} * (1 - \rho_k)))$$

Verify Correctness

- Output image comparison
- Residual convergence validation
- Deterministic execution check: same output on same configuration

Cache Miss per Operation (CMpO)

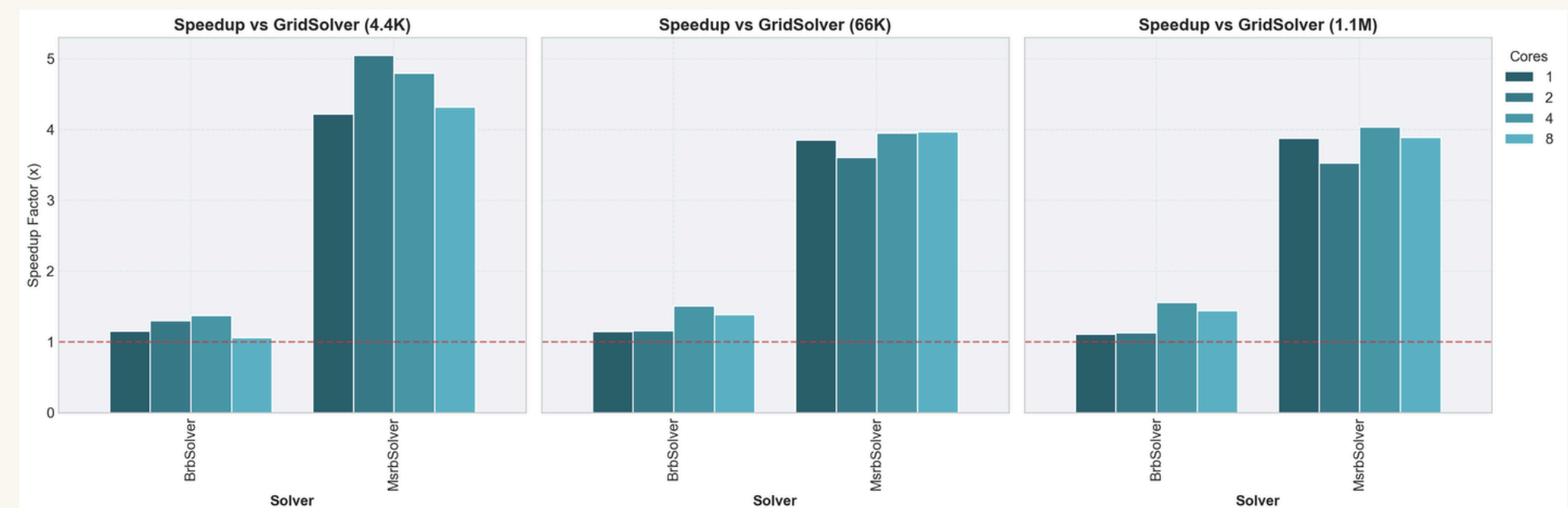
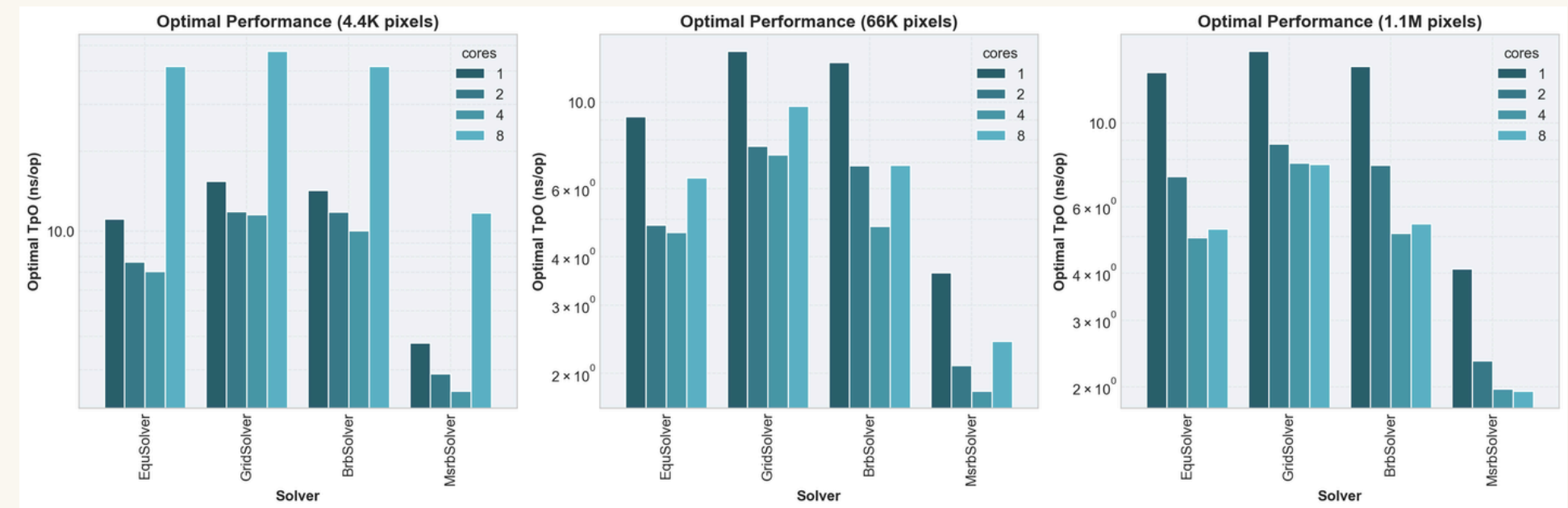
- MsrbSolver: Lowest CMpO across all workloads due to multi-sweep amortization
- BrbSolver: Consistent middle-ground through tile decomposition + in-place updates



As Expected

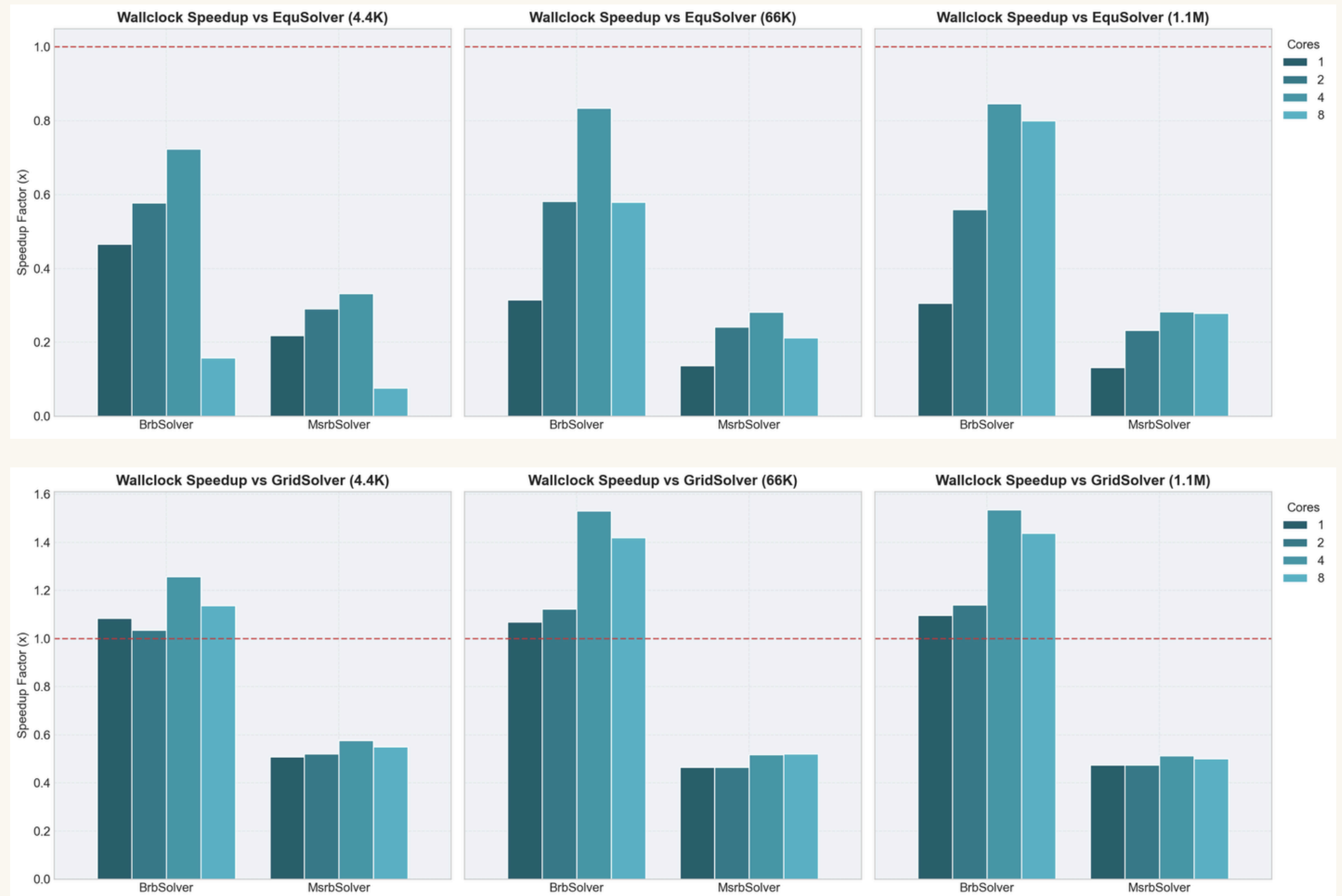
Time per Operation (TpO)

- BrbSolver: Moderate TpO, better scaling at high core counts via tile decomposition
- MsrbSolver: Best overall — multi-sweep amortization hides cache miss latency and improves memory bandwidth utilization



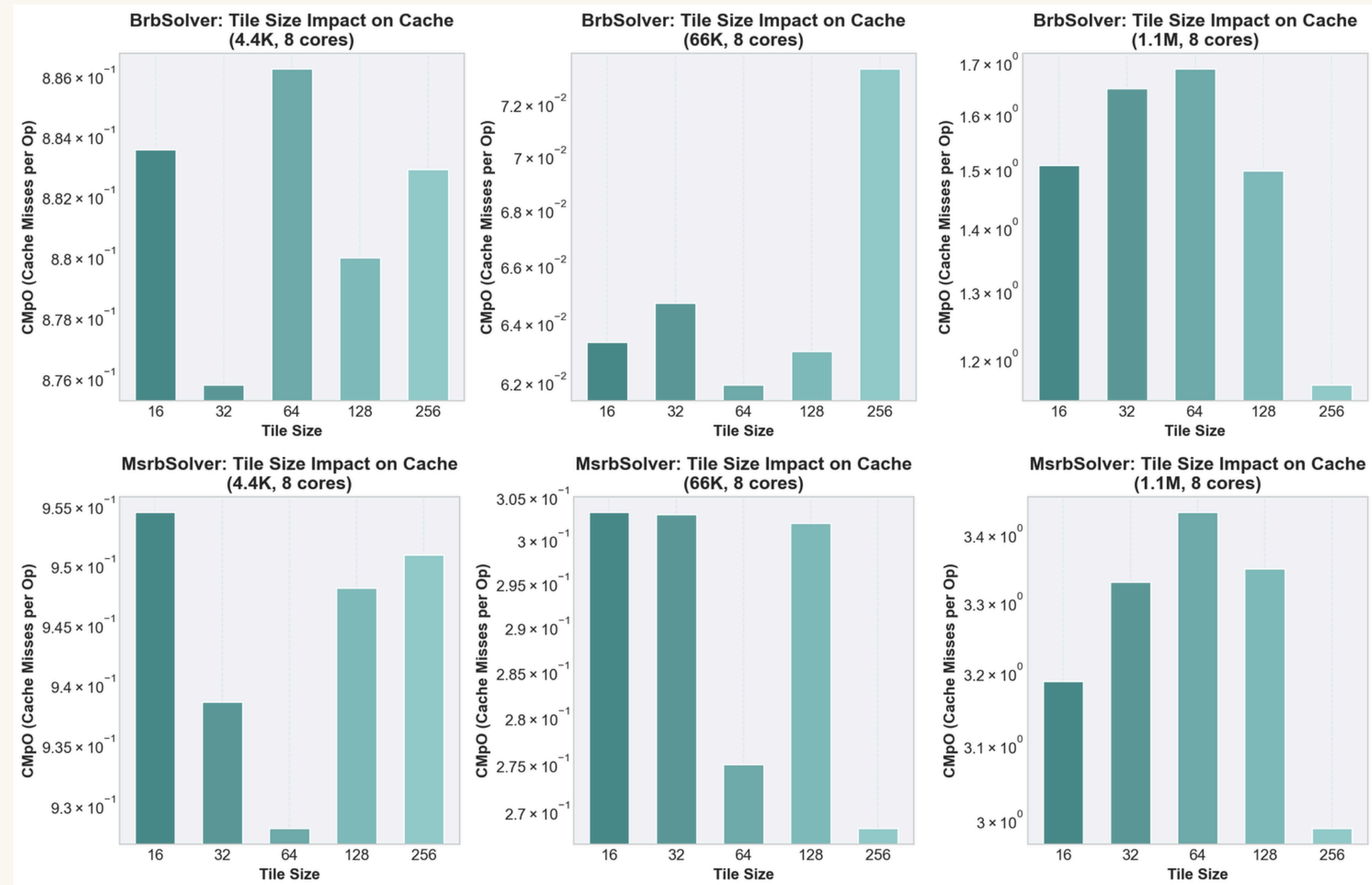
- BrbSolver: Marginal speedup over GridSolver, increasing with core count and input size.
- MsrbSolver: Slowdown 70% to 60% relative to Equ and GridSolver respectively

As Expected



Impact of Grid/Tile Size

- BrbSolver: Optimal tile size scales with workload — 32×32 (small), 64×64 (medium), 128×128 (large)
- MSRB: Consistently prefers larger tiles than BrbSolver (64×64 vs 32×32 on small; 256×256 vs 128×128 on large)



Impact of Alpha for Multi-Sweeps

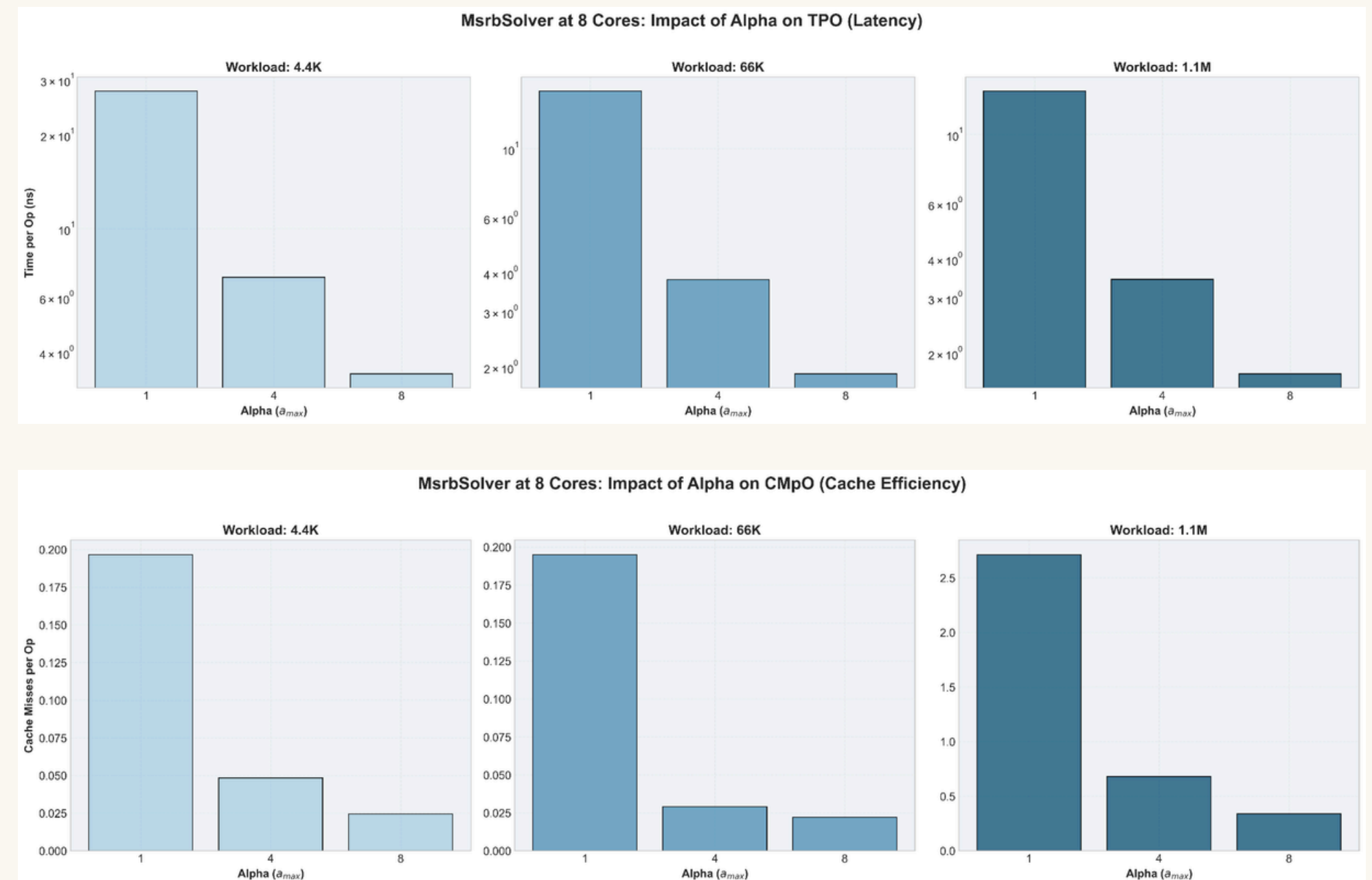
First Sweep loads cache, for remaining sweeps to use

- Remaining sweeps run faster

Higher alpha $\rightarrow$ Better CMpO and Tpo

Higher proportion of sweeps using cached values

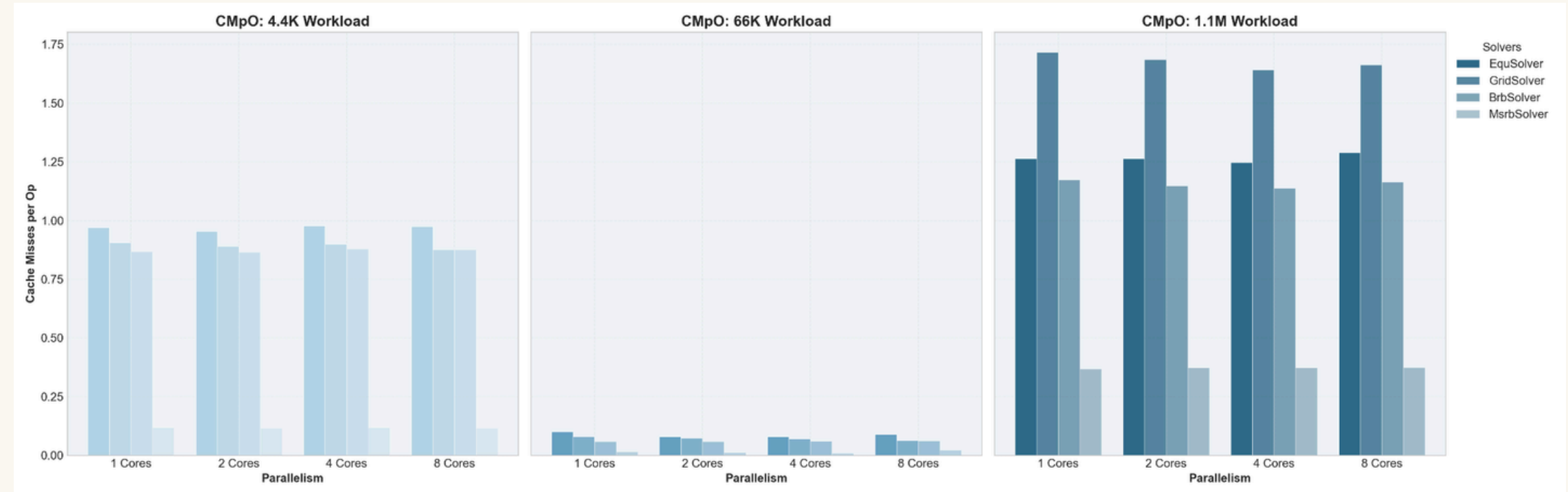
As Expected



Impact of Core Count

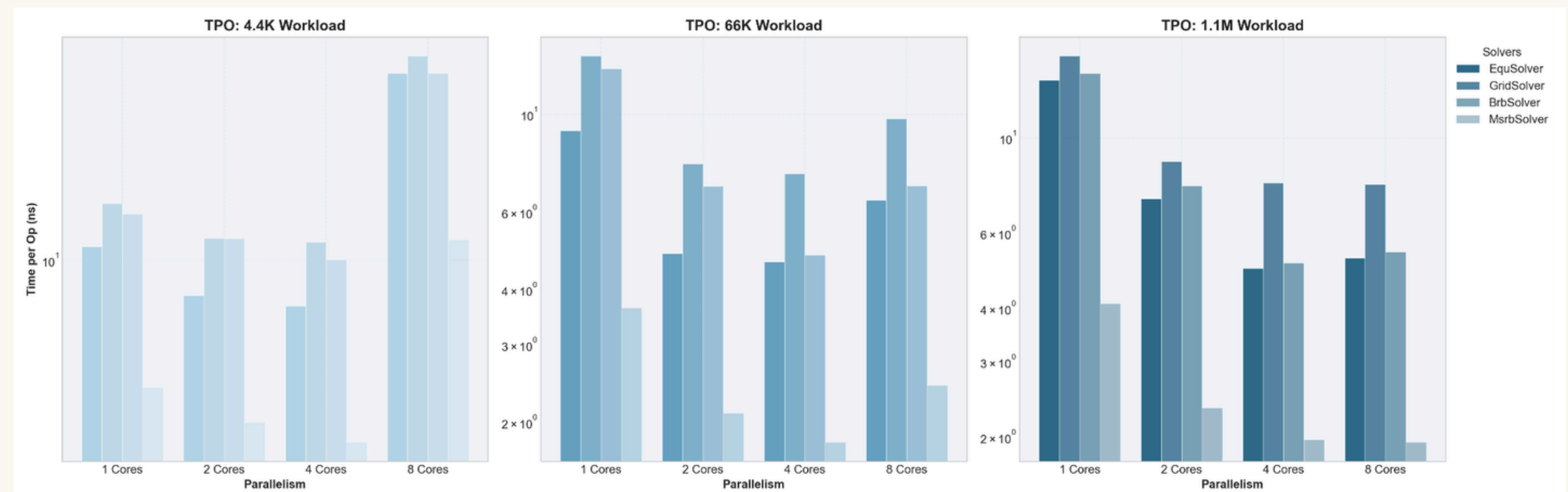
Core Count doesn't affect CMpO

As Expected



Larger workloads benefit from higher parallelism
For all solvers

As Expected



RESULTS

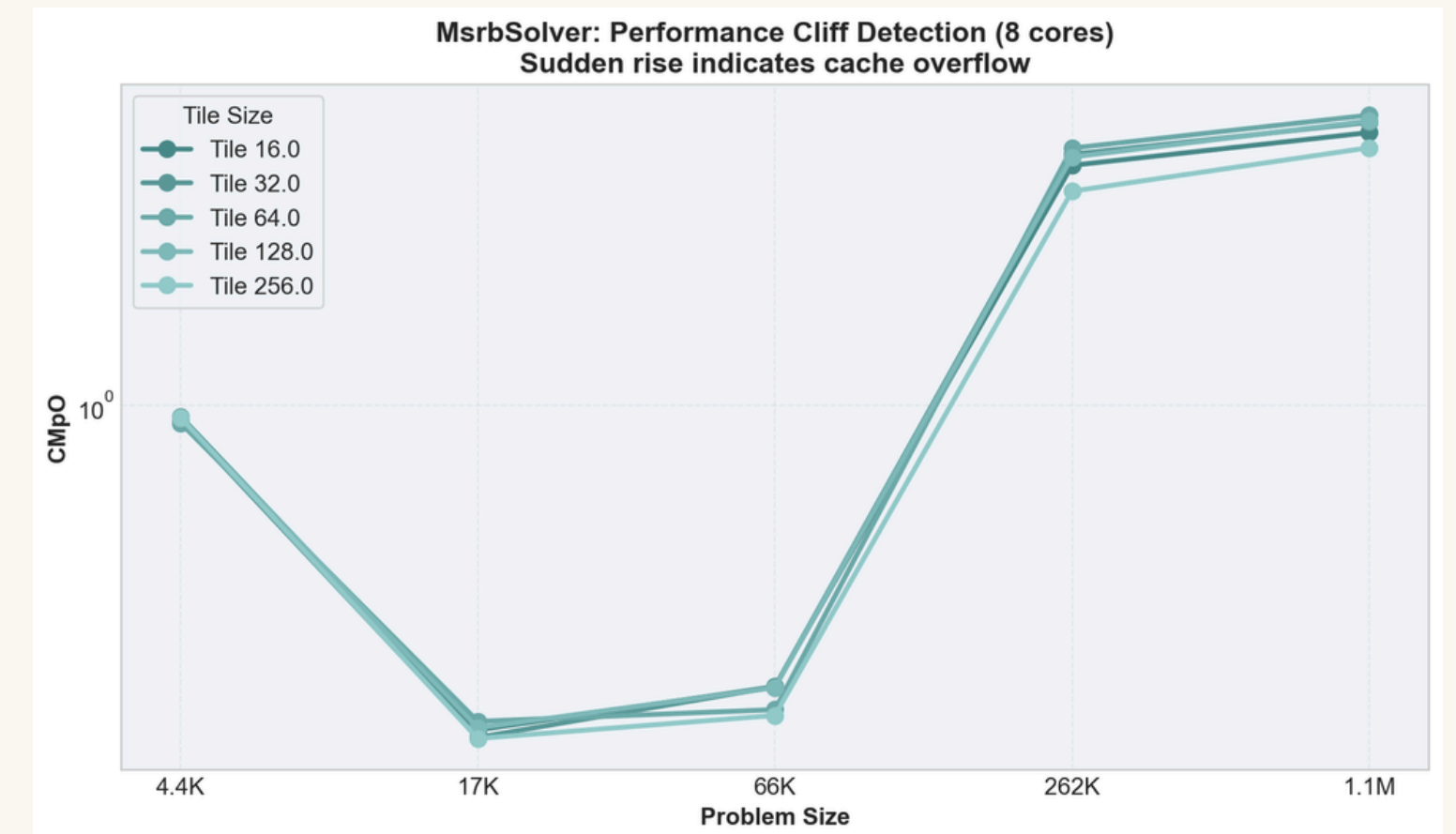
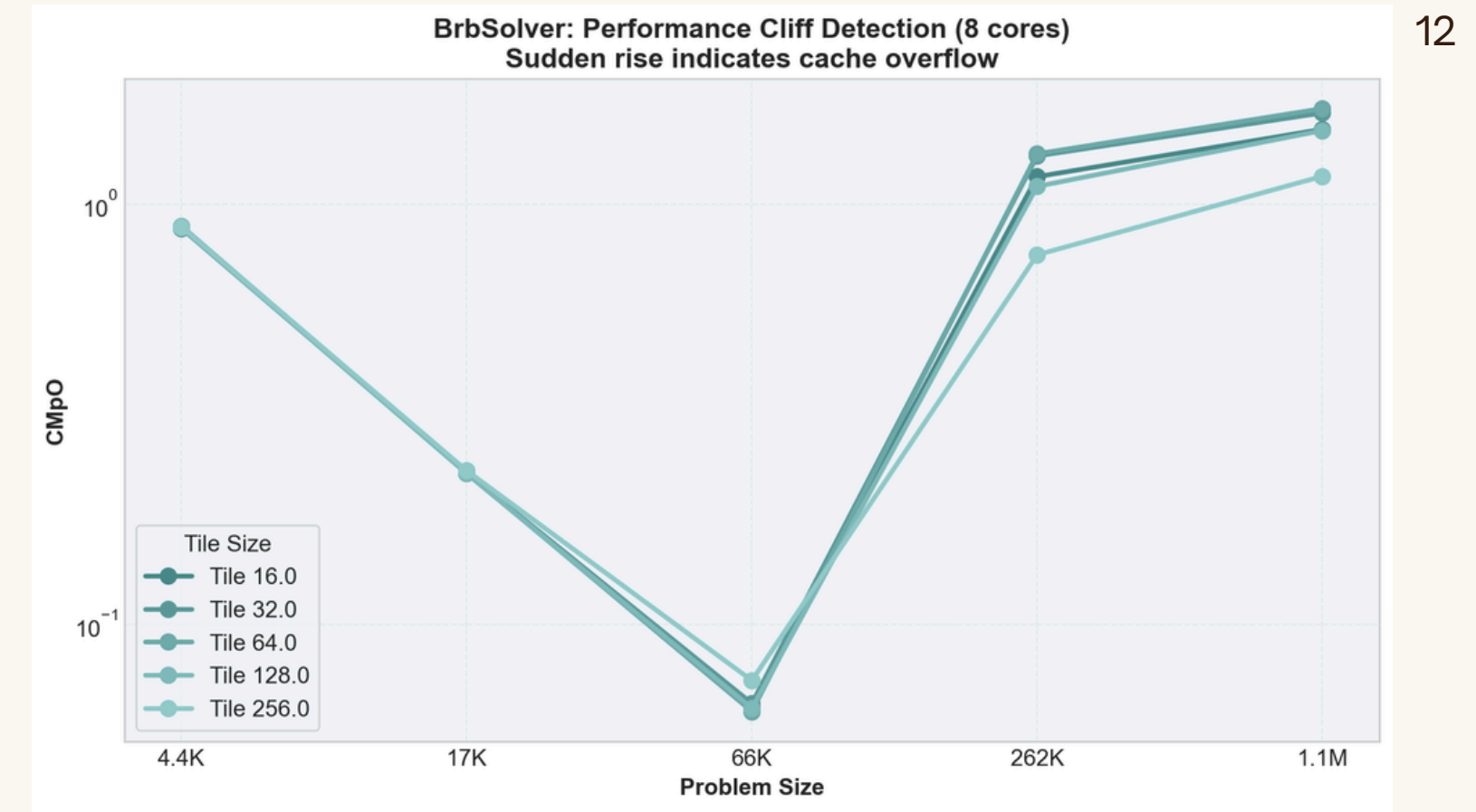
Performance Cliff

Performance Cliff hit after **66k pixels**

Target Machine has 4MiB L3 cache

Thus >66k pixels is >4Mib (8 bits/pixel)

Workload exceeds cache capacity at this point



Residual Error

GridSolver convergences most slowly

- No Gauss Seidel

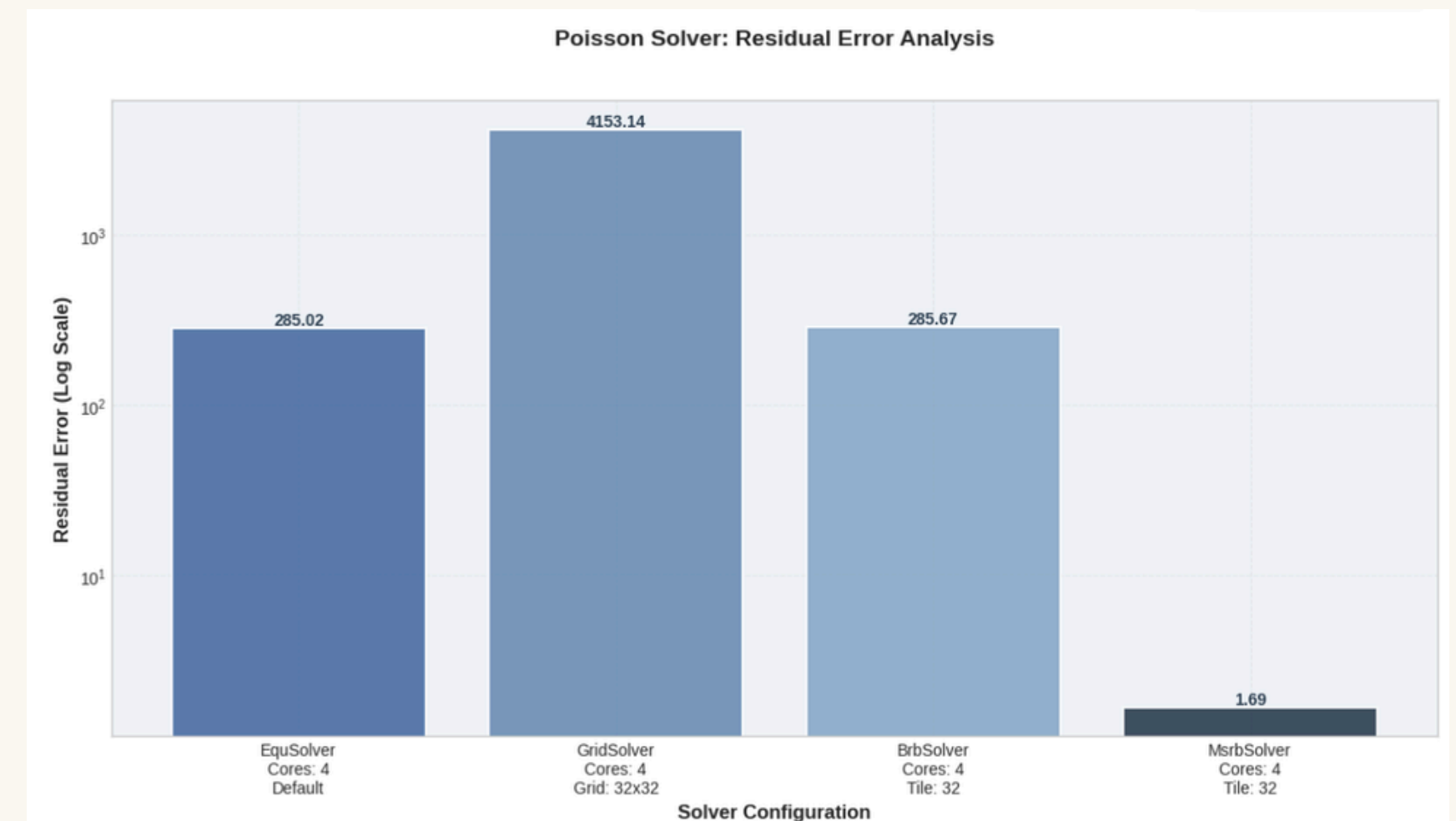
EquSolver matches BrbSolver

MSrb outperforms other solvers:

- 170x smaller residual than EquSolver and BrbSolver
- 2400x smaller residual than GridSolver

Multi-Sweep → more Jacobi updates

As Expected



LLM Usage Disclosure

Usage

- Understanding Poisson Image Editing
- Understanding EquSolver and GridSolver
- Code Generation

Representative Prompts:

Explain how Poisson image editing blends images using gradients to avoid visible seams

Explain EquSolver's red-black coloring: how does it split pixels into two independent folds and why does this enable parallel Gauss-Seidel?"

Benefits

- Helps simplify mathematical concepts
- Help summarise lengthy papers

Limitations

- Can hallucinate and provide inaccurate information
- May overcomplicate the required code during tweaking and deviate from the objective



Conclusion & Key Takeaways

Although both proposed solvers do not give speedup on wall clock time, the observed results align with our predictions

- BrbSolver & MsrbSolver both give improved CMpO due to multi-phase updates
- MsrbSolver has the best CMpO and TpO despite having the slowest wall time
- A higher alpha gives improved CMpO and TpO & residual
- Optimal tile size scales with workload for all solvers
- MsrbSolver consistently prefers larger tiles than BrbSolver

Remained predictable

Due to inflated wall clock time of Msrb, we propose BrbSolver as a strong candidate, to balance cache efficiency and overall speedup.

References

- P. Pérez, M. Gangnet, and A. Blake, “Poisson Image Editing.” [Online]. Available: <https://www.cs.jhu.edu/~misha/Fall07/Papers/Perez03.pdf>
- Trinkle23897, “GitHub - Trinkle23897/Fast-Poisson-Image-Editing: A fast poisson image editing implementation that can utilize multi-core CPU or GPU to handle a high-resolution image input.,” *GitHub*, Mar. 30, 2026.
<https://github.com/Trinkle23897/Fast-Poisson-Image-Editing> (accessed May 07, 2026).
- “Final Report — fpie 0.3.2 documentation,” Readthedocs.io, 2020.
<https://fpie.readthedocs.io/en/main/report.html#> (accessed May 07, 2026).